

第 27 章 autofs 自动挂载

从 map 到触发载荷：把路径模型、按需挂载、空闲卸载和证据链放进同一套判断框架。

对象模型操作语义验证诊断经典任务

大号阅读版

本章阅读导航 先抓住一条主线：**autofs 先建立可触发的路径入口，真正访问 key 时才建立远端载荷；载荷空闲卸载后，入口仍然可以再次触发。** 因此，“配置文件存在”“服务 active”“路径存在”“当前 NFS 已挂载”和“目标身份可读写”必须分别证明。

01 识别路径模型判断 direct 还是 indirect

02 建立 map 关系 master → key → location

03 静态展开确认有效配置视图

04 加载触发层启动或 reload autofs

05 访问 key 触发远端载荷

06 分层验收源、功能、超时与再触发

专题地图 - **知识专题** 从 map 到载荷：autofs 管理的对象链 - **知识专题** indirect 与 direct 的路径模型 - **操作专题** 配置可验证的 indirect NFS 自动挂载 - **知识专题** wildcard `\*` 与替换 `&` - **操作专题** 用 direct map 管理任意绝对路径 - **操作专题** reload、restart、timeout 与 busy - **操作专题** 配置、触发、载荷与功能证据矩阵 - **诊断专题** 路径存在却没有挂载时如何定位 - **经典任务** 用户家目录自动挂载与错误 key 修复

阅读时持续回答 1. master map 与子 map 各自决定什么？ 2. 当前 key 是相对目录名还是完整绝对路径？ 3. `automount -m` 证明了哪一层，尚未证明哪一层？ 4. 当前看到的是 `autofs` 触发层还是 NFS 载荷？ 5. 哪一次精确路径访问会触发挂载？ 6. 超时后载荷消失是故障还是预期生命周期？ 7. 当前挂载源正确后，是否已用目标身份验证功能？ 8. 失败属于 entry、key、trigger、remote、permission 还是 busy？

章节边界：NFS 远端导出与身份语义留给第 26 章；固定挂载与 `/etc/fstab` 留给第 24 章；systemd automount 不作为本章主线替代。

第 27 章 · 正文 网络文件系统并不一定要从开机到关机始终保持挂载。大量用户家目录、偶尔访问的项目共享和低频归档路径，都更适合在真正访问时才连接远端；无人使用一段时间后，载荷可以自然卸载。

`autofs` 正是把这种生命周期表达为配置：master map 建立入口，子 map 描述 key 与远端位置，路径访问触发真实挂载，timeout 管理空闲后的卸载。本章最常见的误判，是把不同状态压成一句“挂载好了”。服务 `active` 只说明 daemon 在运行；`automount -m` 只说明 map 能被读取；父目录或触发路径存在，不代表远端 NFS 载荷此刻已经出现；当前载荷正确，也还没有证明题目指定的用户能够读写。后续所有操作都沿着“配置 → 加载 → 触发 → 当前载荷 → 功能 → 超时 → 再触发”推进，并明确每条证据能证明什么、不能证明什么。

概念

Master Map 是 autofs 的入口索引。它把一个 indirect map 的父挂载点，或者 direct map 使用的 `/-` 标记，关联到具体子 map；它决定“哪些本地路径交给 automounter 管理”，却不直接描述每个远端载荷。看到 master entry 时应继续追踪子 map，而不能把入口配置误认成完整挂载定义。

概念

Indirect Map 在一个共同父目录下按相对 key 形成最终路径。若 master mount point 是 `/rhome`，key 是 `remoteuser0`，触发路径就是 `/rhome/remoteuser0`。子 map 中若误写完整路径，请求 key 将无法匹配；因此 indirect 的核心边界是“父路径由 master 提供，子 map 只写相对 key”。

概念

Direct Map 让子 map 的 key 自身成为完整绝对路径。master map 使用 `/-` 表示 direct 模式，实际路径写在子 map，例如 `/srv/docs`。`/-` 不是要创建或访问的目录；它只是告诉 automounter：不要再把父挂载点与 key 拼接。

概念

Key 是路径访问与 map 条目之间的匹配对象。它在 indirect map 中通常是一个目录组件，在 direct map 中则是完整绝对路径；wildcard `*` 可以接住没有显式条目的请求，而 `&` 把实际请求 key 代入远端位置。判断 key 的含义，是定位“不触发”问题时最有区分度的一步。

概念

触发式挂载 包含两层文件系统：`autofs` 类型的触发层负责拦截路径访问，`nfs/nfs4` 载荷层才是真正的远端文件系统。服务启动后可以已经存在触发层，而在访问 key 前没有任何对应 NFS 载荷；所以“配置存在但尚未挂载”往往是正常初态，而不是错误。

概念

空闲超时 管理的是已触发载荷的生命周期，而不是配置文件的生命周期。载荷在无活动引用并超过 timeout 后可以卸载；若 Shell 的当前目录仍位于其中、文件仍被打开或其他进程仍在使用，载荷可能保持 busy。超时卸载后再次访问能够恢复，才说明按需生命周期完整。

操作语义速查 这一页先建立最关键的接口地图。详细操作仍在后续专题中展开；这里不把所有参数塞进横向表格，而是强调每个入口改变或观察哪一层状态。

`/etc/auto.master` 与 `/etc/auto.master.d/*.autofs`

SYNOPSIS

```
mount-point  map-name  [master-options]
/-           map-name  [master-options]
```

建立 master entry：indirect 模式写父挂载点，direct 模式写 `/-`。它只建立入口和子 map 的关联。

```
/rhome /etc/auto.rhome --timeout=60
```

让 `/rhome` 下的请求交给 indirect map 解析。

```
/- /etc/auto.direct --timeout=60
```

声明 direct map，实际绝对路径写在子 map。

```
*.autofs
```

RHEL 课程环境中用于 `/etc/auto.master.d/` 的本地 master 片段后缀。

direct / indirect 子 map

SYNOPSIS

```
key -mount-options location
```

把请求 key 映射为挂载选项和远端位置。map 类型决定 key 是相对名称还是完整路径。

```
alpha -fstype=nfs,rw host:/exports/alpha
```

indirect 条目；最终路径由 master point 与 `alpha` 拼接。

```
/srv/docs -fstype=nfs,ro host:/exports/docs
```

direct 条目；key 本身就是本地绝对路径。

```
* ... /rhome/&
```

* 匹配请求 key，& 将该 key 代入 location。

```
automount -m
```

SYNOPSIS

```
automount -m
```

展开当前可读取的 master map 与子 map，用于确认 entry、map 来源和 key 是否进入有效配置视图。

```
-m
```

打印 maps 后退出；属于静态配置证据。

证明边界

不证明服务已应用新配置，不证明远端可访问，也不证明 key 已经触发。

```
systemctl reload/restart autofs
```

SYNOPSIS

```
systemctl enable --now autofs
systemctl reload autofs
systemctl restart autofs
```

控制 daemon 当前状态、启动持久性和配置重新读取。修改 map 后优先走“静态展开 → reload → 重新触发”。

`enable --now`

首次部署时同时建立当前运行和开机启用状态。

`reload`

重新读取配置，尽量保留现有 daemon 生命周期。

`restart`

完整重启服务，影响更强；不能修复错误 key 或远端路径。

stat / ls -ld

SYNOPSIS

```
stat /base/key
ls -ld /base/key
```

对精确 key 发起路径解析，从而有意识地触发按需挂载。只列父目录不一定枚举所有尚未触发的 key。

`stat /rhome/remotouser0`

直接访问目标 key，适合触发与错误复现。

`ls -ld PATH`

查询目标路径本身，避免普通 `ls PATH` 继续遍历大量内容。

findmnt

SYNOPSIS

```
findmnt -t autofs
findmnt -t nfs,nfs4
findmnt -T /base/key
```

读取当前挂载关系，用于区分触发层与远端载荷，并核对某个路径当前实际落在哪个文件系统上。

`-t autofs`

观察受管理的触发层。

`-t nfs,nfs4`

观察当前已经触发的 NFS 载荷。

`-T PATH`

从具体路径反查覆盖它的挂载源、目标与类型。

[知识专题] 从 map 到载荷：autofs 管理的到底是什么

理解 autofs 的最短路径不是先背三行配置，而是先画出对象链。master map 决定哪些路径由 automounter 管理；子 map 决定某个 key 对应什么文件系统；内核 autofs 文件系统负责在路径访问时通知用户态 daemon；daemon 再执行真实挂载。排错时必须知道证据位于哪一层，否则容易把“目录已经出现”误判为“NFS 已经挂上”，或把“服务 active”误判为“远端读写已经成功”。

① [知识点] master map 是入口索引，子 map 才描述具体载荷

master map 的基本形式是：

```
mount-point map-name options
```

例如：

```
/rhome /etc/auto.rhome --timeout=60
```

这一行只说明 `/rhome` 由 `/etc/auto.rhome` 管理，并为该 map 设置 60 秒空闲超时。它没有直接给出某个用户目录来自哪台服务器。具体 key、挂载选项和远端源在子 map 中定义：

```
remoteuser0 -fstype=nfs,rw serverb.lab.example.com:/rhome/remoteuser0
```

因此排错时应分开问：master entry 是否被加载，子 map 是否可读，目标 key 是否存在或能被 wildcard 匹配，远端 location 是否正确。

② [知识点] key 是访问路径与 map 条目之间的匹配对象

对 indirect map，key 通常是一个目录组件。master map 中的父路径为 `/rhome`，key 为 `remoteuser0`，最终触发路径就是：

```
/rhome/remoteuser0
```

访问 `/rhome/remoteuser0/project/file` 时，autofs 首先用 `remoteuser0` 查找 map，然后把剩余子路径交给已经挂载的远端文件系统。把完整路径 `/rhome/remoteuser0` 错写成 indirect key，会使请求的 key 与 map 不匹配。

对 direct map，key 则是完整绝对路径。本章后面会单独建立 direct/indirect 的比较模型。

③ [知识点] 触发层与载荷层必须分开观察

服务加载 indirect map 后，系统会建立一个 `autofs` 类型的触发文件系统。它负责拦截对 key 的访问，但不等于远端 NFS 已经挂载。触发前可能出现以下状态：

```
findmnt -t autofs      # 能看到受管理的触发层
findmnt -t nfs,nfs4    # 看不到目标远端载荷
```

对精确 key 执行 `stat /rhome/remoteuser0` 后，才应在当前挂载关系中看到对应 NFS 源。空闲超时后，NFS 载荷可以消失；只要触发层与 map 仍有效，再次访问就应重新挂载。

④ [知识点] 状态链比一条“成功命令”更重要

本章采用以下状态链：

```
配置文件存在
→ master 与子 map 可解析
→ autofs.service 当前运行
→ 触发层已建立
→ 精确 key 能触发
→ 当前载荷源正确
→ 目标身份完成所需读写
→ 空闲后可卸载
→ 再次访问可恢复
```

`automount -m` 只能推进到“map 可解析”；`systemctl is-active` 只能证明 daemon 当前运行；`findmnt -T` 只能证明当前路径对应哪个已挂载文件系统；只有按题目身份完成访问，才能证明业务功能层。

⑤ [知识点] idle、busy 与 expire 构成自动卸载模型

超时并不是从“配置完成”开始计算，而是与载荷的实际使用和空闲状态有关。如果进程的当前工作目录位于挂载中、文件仍被打开、或存在其他活动引用，载荷可能保持 busy，不能在预期时间卸载。此时“超过 60 秒仍存在”不应立即解释为 timeout 配置无效，应先确认是否真正空闲。

超时卸载的价值不是追求每次都立即消失，而是让不再使用的网络挂载能够自然释放。考试和日常验证时，先关闭使用该目录的 Shell 或程序，再等待超时并查询当前 NFS 载荷；不要默认使用强制卸载破坏仍在使用的进程。

[Cheatsheet] master map 定义入口，子 map 定义 key 与载荷；autofs 触发层不等于 nfs 载荷；active、可解析、已触发和可读写要分层验证；超时前先确保没有活动引用。

[知识专题] indirect 与 direct：路径是怎样由 key 形成的

两类 map 使用同样的“key -options location”语法，但 key 的含义不同。indirect map 适合在同一个父目录下管理一组同类路径，例如用户家目录或项目目录；direct map 适合把若干完整路径直接映射到远端源。最容易出现的错误是把 direct 的绝对路径写法放进 indirect map，或在 master map 中忘记 direct map 必须使用 `/-`。

① [知识点] indirect map 使用“父挂载点 + 相对 key”

示例：

```
# /etc/auto.master.d/projects.autofs
/projects /etc/auto.projects --timeout=60

# /etc/auto.projects
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
beta -fstype=nfs,rw files.example.com:/exports/projects/beta
```

最终路径分别是：

```
/projects/alpha  
/projects/beta
```

子 map 中只写 `alpha` 和 `beta`，不写 `/projects/alpha`。master mount point 与 key 的拼接关系就是 indirect map 的核心。

② [知识点] direct map 使用 `/-` 与完整绝对路径 key

示例：

```
# /etc/auto.master.d/direct.autofs  
/- /etc/auto.direct --timeout=60  
  
# /etc/auto.direct  
/srv/docs -fstype=nfs,rw files.example.com:/exports/docs  
/opt/catalog -fstype=nfs,ro files.example.com:/exports/catalog
```

`/-` 不是实际目录，也不是要访问的路径；它是 master map 中的 direct map 标记。真正的本地挂载点写在子 map 的 key 中。direct map 可以在文件系统层次的不同位置管理多个完整路径。

③ [知识点] map 一行包含 key、选项和 location

子 map 基本形式是：

```
key -options location
```

对于 NFS：

```
key -fstype=nfs,rw host:/export/path
```

选项字段以 `-` 开头，多个选项以逗号分隔。`location` 使用 NFS 的 `host:/path` 表示法。NFS 是 autofs 常见默认类型，但在考试答案和教学示例中显式写 `-fstype=nfs,rw` 能减少对象歧义；题目若指定只读或其他挂载选项，应按要求替换，而不是机械复制 `rw`。

④ [知识点] master map 选项与子 map 选项作用层次不同

master entry 的选项通常影响整个 map，例如：

```
/projects /etc/auto.projects --timeout=60
```

子 map 选项描述具体载荷，例如：


```
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
```

不要把 `--timeout=60` 写进 NFS mount options，也不要把 `-fstype=nfs,rw` 写成 master map 的文件路径。阅读配置时先按字段位置判断作用对象。

⑤ [知识点] `/etc/auto.master.d/*.autofs` 适合放本地扩展入口

RHEL 9 的课程环境常在 `/etc/auto.master.d/` 中创建以 `.autofs` 结尾的 master map 片段，例如：

```
/etc/auto.master.d/rhome.autofs
```

这样能把本地任务与主文件中的 vendor 或站点配置分开。文件扩展名错误、路径拼错或子 map 文件不存在，都会使预期 entry 不进入有效配置。无论写入 `/etc/auto.master` 还是 drop-in，最终都要用 `automount -m` 确认实际加载结果，而不是只相信编辑器中的内容。

⑥ [比较点] 选择 direct 还是 indirect 的判断方法

优先问本地路径是否共享一个自然父目录：

- `/rhome/alice`、`/rhome/bob`、`/rhome/carol`：适合 indirect map；
- `/srv/docs` 与 `/opt/catalog`：没有共同业务父目录，适合 direct map；
- 题目明确给出“在 `/base/key` 下按 key 挂载”：使用 indirect；
- 题目要求几个任意绝对路径分别按需挂载：使用 direct。

选择 map 类型是在表达本地路径模型，而不是在选择 NFS 版本。

[Cheatsheet] indirect：master point + relative key；direct：master 使用 `/-`，子 map key 为完整路径；子 map 行是 `key -options location`；`.autofs` 片段必须由有效 master 配置加载。

[操作专题] 配置一个可验证的 indirect NFS 自动挂载

本专题建立考试中最常见的完整流程：远端 NFS 已由第 26 章准备，本机需要把用户或项目目录按 key 自动挂载。操作不能从直接写配置开始，应先确认不会与已有 entry 冲突，再建立 master 和子 map，静态展开、加载、精确触发，并用目标身份完成功能验证。

① [操作] 调查现状并确认前置对象

作用对象：本机已有 autofs 配置、服务状态和目标路径。

基本语义：先建立基线，避免重复 master entry、覆盖站点配置或把远端问题误当成本地 map 问题。

```
rpm -q autofs nfs-utils
systemctl is-active autofs
systemctl is-enabled autofs
automount -m
findmnt -t autofs
findmnt -t nfs,nfs4
ls -ld /rhome
```

需要 NFS 服务器名称时，可使用第 26 章的名称解析和远端源验证方法。这里不要求为了配置 autofs 先做一个永久手工挂载；若为了诊断临时挂载，完成后必须卸载，最终终态仍由 autofs 提供。

② [操作] 安装客户端组件并创建 master 片段

作用对象：autofs 和 NFS 客户端软件包、master map 入口。

```
dnf install autofs nfs-utils

cat > /etc/auto.master.d/rhome.autofs <<'EOF'
/rhome /etc/auto.rhome --timeout=60
EOF
```

这里显式设置 60 秒仅用于任务与练习，使超时验证可控；不要把它推断为所有系统的默认值。若目标目录已经存在，应先检查其中是否有本地文件。挂载后这些内容可能被覆盖在载荷下面，不能用“目录原来是空的”作为无条件假设。

③ [操作] 创建具体 key 或 wildcard 子 map

只管理一个用户时：

```
cat > /etc/auto.rhome <<'EOF'
remoteuser0 -fstype=nfs,rw serverb.lab.example.com:/rhome/remoteuser0
EOF
```

管理同名用户目录族时：

```
cat > /etc/auto.rhome <<'EOF'
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
EOF
```

map 文件应保持普通文本格式。不要在字段之间混入全角空格，不要把 shell 引号写进最终 map，不要把整条配置当成命令执行。

④ [操作] 用 automount -m 做静态展开

作用对象：当前可读取的 master map 与子 map。

```
automount -m
```

应在展开结果中确认：

```
master mount point: /rhome
map source: /etc/auto.rhome
key: remoteuser0 或 wildcard
location: serverb.lab.example.com:/rhome/...
timeout: 与任务要求一致
```

此时还不能声明远端已挂载。`automount -m` 不测试远端服务器是否可达，也不证明用户具有写权限；它的主要价值是尽早发现 master 未加载、子 map 不存在、字段结构错误和 key 写错。

⑤ [操作] 首次启用或重新加载服务

首次部署：

```
systemctl enable --now autofs.service
```

服务已经运行且修改了配置：

```
systemctl reload autofs.service
```

随后分别检查：

```
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
```

`active` 与 `enabled` 是两个维度。`reload` 成功也只说明 daemon 接受了重新读取请求，仍需触发目标 key。

⑥ [操作] 精确触发并核对当前源

不要只执行 `ls /rhome`。父目录是否显示潜在 key 受 `browse` 行为和 `map` 类型影响；直接访问精确路径更可靠：

```
stat /rhome/remoteuser0
# 或
ls -ld /rhome/remoteuser0
```

触发后查询：

```
findmnt -T /rhome/remoteuser0  
findmnt -t nfs,nfs4
```

核对的不是“有一条 NFS”即可，而是本地目标、远端主机和导出路径都与题目一致。

⑦ [验证] 用题目身份验证读写，而不是用 root 代替

```
sudo -u remoteuser0 test -r /rhome/remoteuser0  
sudo -u remoteuser0 touch /rhome/remoteuser0/.autofs-client-test  
sudo -u remoteuser0 rm /rhome/remoteuser0/.autofs-client-test
```

如果题目只要求只读，则不要创建文件；使用对应身份读取一个已知对象。root 写成功不能证明普通用户可写，普通用户失败也不能靠本地 `chmod 777` 自动修复远端导出和 UID/GID 问题。

⑧ [验证] 检查空闲卸载与重新触发

退出位于目标目录中的 Shell，关闭引用该目录的程序，然后等待超过显式 timeout：

```
cd /  
# 等待超过任务设定的空闲超时  
findmnt -t nfs,nfs4
```

对应载荷消失后，再次执行：

```
stat /rhome/remoteuser0  
findmnt -T /rhome/remoteuser0
```

再次出现正确源，才证明“按需挂载—空闲卸载—再触发”的生命周期闭环。静态环境中只能给出推荐验证步骤，不能声称已经观察到实际时间行为。

[Cheatsheet] 基线 → master drop-in → 子 map → `automount -m` → enable/reload → `stat` 精确触发 → `findmnt -T` → 目标身份读写 → 空闲后再触发。

[知识专题] wildcard 与 &：让请求 key 进入远端路径

逐个列出所有用户或项目 key 并不总是可维护。当本地 key 与远端末级目录同名，并且所有条目使用相同选项时，可以用 wildcard map 表达一个规则。* 与 & 的关系很简单，但要记住它只是字符串匹配与替换，不会预先检查远端目录是否存在。

① [知识点] * 匹配未被具体条目定义的请求 key

```
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
```

访问 `/rhome/remotouser0` 时，请求 key 是 `remotouser0`，wildcard 条目被选中。访问 `/rhome/alice` 时，请求 key 变成 `alice`。wildcard 让一个 map 覆盖同构目录族，但也扩大了可尝试的 key 范围。

② [知识点] `&` 在 location 中代入实际 key

对请求 key `remotouser0`，上面的 location 解析为：

```
serverb.lab.example.com:/rhome/remotouser0
```

`&` 不是 shell 变量，不需要 `$`，也不是引用 master mount point。它表示当前匹配的 key。把远端位置写成固定 `/rhome/remotouser0` 会使所有请求都指向同一目录；漏写 `&` 则失去同名映射能力。

③ [边界] wildcard 不证明远端同名目录真实存在

`automount -m` 能显示 wildcard 规则，却不会为每个可能的 key 访问远端服务器。访问一个不存在的 key 时，触发可能失败，并且失败查找可能被短暂缓存。诊断时应分别检查：请求 key 是否正确、替换后的远端路径是否存在、NFS 服务器是否允许客户端访问。

④ [比较点] 具体 key 与 wildcard 的使用边界

只有一个考试用户时，具体 key 更直观：

```
remotouser0 -fstype=nfs,rw serverb:/rhome/remotouser0
```

题目明确需要“所有同名用户目录”或希望表达可扩展规则时，wildcard 更合适：

```
* -fstype=nfs,rw serverb:/rhome/&
```

不要为展示技巧而把一个固定、异名映射强行改为 wildcard；map 应忠实表达题目对象关系。

[Cheatsheet] `*` 捕获请求 key，`&` 代入该 key；规则可解析不代表远端目录存在；具体对象用具体 key，同名目录族再用 wildcard。

[操作专题] 用 direct map 管理任意绝对路径

当需要按需挂载的本地路径分散在不同父目录下时，建立多个只有一个 key 的 indirect map 会增加结构复杂度。direct map 允许在一个子 map 中直接列出完整本地路径。它的核心不是“更高级”，而是 key 的路径语义发生了变化。

① [操作] 建立 direct master entry

```
cat > /etc/auto.master.d/direct.autofs <<'EOF'
/- /etc/auto.direct --timeout=60
EOF
```

/- 必须出现在 master map 的 mount-point 字段。不要把实际路径 `/srv/docs` 写在这一字段并同时在 direct map 中再次写完整路径；那会混合两种模型。

② [操作] 在子 map 中使用完整绝对路径 key

```
cat > /etc/auto.direct <<'EOF'
/srv/docs -fstype=nfs,rw files.example.com:/exports/docs
/opt/catalog -fstype=nfs,ro files.example.com:/exports/catalog
EOF
```

确保父目录层次不会与已有关键本地数据冲突。automounter 可以创建需要的挂载目录，但在生产变更前仍应调查目标路径是否已经被其他文件系统、服务或应用使用。

③ [操作] 展开、reload 与逐路径触发

```
automount -m
systemctl reload autofs.service

stat /srv/docs
findmnt -T /srv/docs

stat /opt/catalog
findmnt -T /opt/catalog
```

一个 direct map 中的两个 key 是两个独立触发对象；第一个成功不能证明第二个 location 也正确。

④ [诊断点] direct map 修改后优先显式 reload

上游 autofs 语义要求 direct map 和 master map 的变化刷新 daemon。为保证考试和 workflows 确定性，本章对 direct map 采用：

```
修改 → automount -m → systemctl reload autofs → 精确访问 → findmnt
```

不要依赖“下次操作也许自动识别”的模糊预期。若 entry 当前正在使用，某些变更可能要等旧载荷过期后才完全体现：先退出引用路径的进程，再进行最小变更和复验。

⑤ [比较点] direct 与 indirect 的错误输出判断

若 master 使用 `/projects /etc/auto.projects`，子 map 的 key 应是 `alpha`；若 master 使用 `/- /etc/auto.direct`，子 map 的 key 应是 `/projects/alpha`。看到“map 能展开但访问

`/projects/alpha` 无载荷”时，首先比较 master 类型与 key 形式，而不是直接修改远端服务器。

[Cheatsheet] direct master entry 固定用 `/-`；子 map key 写完整绝对路径；每个路径独立触发；direct/master 变化采用显式 reload。

[操作专题] reload、restart、timeout 与 busy：控制配置和生命周期

服务控制不是“改完配置一律 restart”。reload 的目标是让运行中的 automounter 重新比较 master map、刷新相应 map，并尽量保留仍在使用的状态；restart 会停止并重新启动服务，影响更大。超时则决定空闲载荷何时尝试卸载。把三者分开，才能在不扩大影响的情况下完成变更。

① [操作] 首次部署建立当前与持久状态

```
systemctl enable --now autofs.service
systemctl is-active autofs.service
systemctl is-enabled autofs.service
```

`enable --now` 同时请求当前启动和开机启用，但验证仍要分开。服务 active 只说明 automount daemon 正在运行，不说明每个 map、key 和远端 location 都正确。

② [操作] 修改配置后优先使用 reload

```
automount -m
systemctl reload autofs.service
systemctl status autofs.service --no-pager
```

普通 indirect map 的变更通常可以在后续查找中被识别，但 master map、direct map 以及影响浏览行为的变更需要显式 reload。为了形成统一、可复现的考试流程，本章在修改任何本地 map 后都建议先静态展开再 reload。

③ [边界] restart 是更强操作，不是解析错误的替代品

```
systemctl restart autofs.service
```

restart 可用于 daemon 状态异常、课程步骤明确要求，或 reload 无法建立预期管理关系时。执行前应确认当前载荷和使用者，因为停止服务可能与 busy mount 发生冲突。若 `automount -m` 已经显示语法错误，restart 只会让错误再次加载，不会修复配置。

④ [参数] 在 master entry 中显式设置 timeout

```
/rhome /etc/auto.rhome --timeout=60
```

timeout 以秒为单位描述空闲载荷尝试卸载前的时间。上游程序默认值与发行版安装配置可能不同，因此任务需要可观察超时时应显式设置，不制作“所有 RHEL 9 必定为某个默认秒数”的背诵结论。

将 timeout 设为 0 会禁用自动卸载。这是配置边界，不是普通考试答案；若题目要求“空闲后自动卸载”，0 显然不满足终态。

⑤ [诊断点] busy 载荷不会因计时到达而安全消失

超时未卸载时，先检查最常见的活动引用：

```
pwd
findmnt -T /rhome/remotouser0
fuser -vm /rhome/remotouser0
```

另一个 Shell 的当前目录、打开文件或后台进程都可能保持 busy。先让使用者退出或关闭文件，再重新等待；不要在没有调查时使用强制卸载。强制解除仍被进程使用的挂载可能造成 I/O 错误或业务异常。

⑥ [验证] reload/restart 后仍要回到功能证据

无论控制命令是否成功，最终都要执行：

```
automount -m
→ systemctl is-active/is-enabled
→ findmnt -t autofs
→ stat 精确 key
→ findmnt -T 目标路径
→ 目标身份读写
```

服务控制结果只是中间证据。把 `systemctl reload` 返回成功直接写成“自动挂载完成”，属于典型的终态扩大。

[Cheatsheet] 首次部署 `enable --now`；修改后“`automount -m + reload`”；restart 影响更强；timeout 管空闲载荷；未卸载先查 busy，不默认强制。

[操作专题] 建立配置、触发、载荷与功能的证据矩阵

验证的目标不是堆命令，而是让每条命令回答一个不同问题。autofs 尤其容易出现“触发前无 NFS 结果是正常的”“触发层存在但载荷失败”“载荷成功但目标用户不能写”等看似矛盾的状态。下面的矩阵给出一条从静态到动态、从本机到远端功能的验收路径。

① [验证点] `automount -m` 证明有效配置视图

```
automount -m
```

检查 master mount point、map 文件、key、选项、location 和 timeout。它不能证明：

- autofs.service 当前已经应用该配置；
- 服务器名称可解析；
- NFS 导出允许本机访问；
- 目标用户具有读写权限。

因此它是第一层，而不是最终层。

② [验证点] `systemctl` 分开回答 `current` 与 `persistent`

```
systemctl is-active autofs.service
systemctl is-enabled autofs.service
systemctl status autofs.service --no-pager
```

`is-active` 回答 daemon 现在是否运行，`is-enabled` 回答系统启动时是否配置为启用。`status` 适合查看最近状态和错误摘要，但不能替代 `map` 展开与功能访问。

③ [验证点] `findmnt -t autofs` 观察触发层

```
findmnt -t autofs
```

它用于确认 automounter 已为 master entry 建立管理层。看到 `/rhome` 的 `autofs` 触发层不等于 `/rhome/remoteuser0` 的 NFS 已经挂载；反过来，目标载荷超时消失后，触发层仍可能保留。

④ [验证点] 精确访问触发 key

```
stat /rhome/remoteuser0
# 或
ls -ld /rhome/remoteuser0
```

精确访问比只列父目录更有区分度。`ls /rhome` 是否展示未触发 key 可能受 `browse` 行为影响，不能把父目录为空直接判为 `map` 错误。

⑤ [验证点] `findmnt -T` 核对目标路径的真实来源

```
findmnt -T /rhome/remoteuser0
```

触发后，应核对 `TARGET`、`SOURCE`、`FSTYPE` 和 `OPTIONS`。`findmnt -T` 会解析给定路径，可能本身引发路径查找，因此不要把它作为“绝对不会触发”的前置观察命令；当需要明确触发时，先使用 `stat`，再把 `findmnt -T` 用于源关系验收。

⑥ [验证点] 当前 NFS 列表用于比较触发前后和超时而

```
findmnt -t nfs,nfs4 -o TARGET,SOURCE,FSTYPE,OPTIONS
```

同一任务可以记录三个时刻：

触发前：目标载荷尚不存在
触发后：目标载荷与远端源出现
空闲后：目标载荷消失
再次访问：目标载荷重新出现

不要要求文件系统类型一定显示为 `nfs` 而不是 `nfs4`，除非题目明确指定版本；关键是源、目标和功能正确。

⑦ [验证点] `ls/stat` 只能证明路径访问，目标身份才证明权限

`stat` 成功说明路径可解析并能读取元数据，但不一定证明题目身份能创建文件。完整功能测试应按要求使用实际用户：

```
sudo -u remoteuser0 test -r /rhome/remoteuser0  
sudo -u remoteuser0 test -w /rhome/remoteuser0
```

若需要创建测试文件，使用唯一、可清理的名称，并在验证后删除。不要假设 `root` 的结果代表普通用户。

⑧ [验证点] `journal` 是运行证据，不是孤立的“答案生成器”

```
journalctl -u autofs.service -b --no-pager
```

日志适合确认 `reload`、`key lookup`、`mount`、`expire` 和错误消息的时间顺序。先用 `map`、服务和挂载证据缩小层次，再读相应时间段；不要从一条模糊日志直接跳到关闭 SELinux、修改服务端权限或重装软件。

[Cheatsheet] `automount -m` 看配置；`is-active/is-enabled` 看服务两维；`findmnt -t autofs` 看触发层；`stat` 触发；`findmnt -T` 核对源；目标身份验证功能；`journal` 解释运行过程。

[诊断专题] 路径存在却没有挂载：按证据定位失败层

“访问失败”并不是一个根因。`autofs` 的故障至少可落在配置加载、key 匹配、触发、远端 NFS、身份权限和过期回收六层。稳定诊断遵循：症状 → 当前证据 → 假设 → 下一条最有区分度的证据 → 最小修复 → 再验证。

① [诊断点] `automount -m` 中没有目标 `master entry`

症状：配置文件已经创建，但有效 `map` 中看不到 `/rhome` 或 `/-` `entry`。

当前假设： `.autofs` 后缀错误、文件放错目录、`master` 片段语法错误、`map` 路径拼错或 `entry` 被其他配置冲突。

下一条证据：核对文件名、内容和 `automount -m` 的错误信息：

```
ls -l /etc/auto.master /etc/auto.master.d
sed -n '1,120p' /etc/auto.master.d/rhome.autofs
automount -m
```

最小修复：只修正入口文件名或对应字段，重新展开并 `reload`；不要删除所有已有 `autofs` 配置。

② [诊断点] `master` 可见，但具体 `key` 不出现或不触发

症状：`/projects` 已由 `autofs` 管理，访问 `/projects/alpha` 没有对应 NFS 载荷。

当前假设：`indirect key` 错写成完整路径、`key` 拼写错误、`wildcard` 规则不正确或子 `map` 未被读取。

下一条证据：比较 `master` 类型、请求 `key` 与子 `map` 第一字段：

```
automount -m
sed -n '1,120p' /etc/auto.projects
stat /projects/alpha
journalctl -u autofs.service -b --no-pager
```

若 `master` 是 `/projects /etc/auto.projects`，正确 `key` 应为 `alpha`，不是 `/projects/alpha`。

③ [诊断点] 目录存在，但 `findmnt` 没有远端载荷

症状：`ls -ld /rhome` 成功，用户据此认为已经挂载。

判断：目录存在只证明路径节点存在。它可能是普通本地目录，也可能是 `autofs` 触发层，或只是载荷超时后留下的可触发入口。

下一条证据：

```
findmnt -t autofs
findmnt -t nfs,nfs4
stat /rhome/remoteuser0
findmnt -T /rhome/remoteuser0
```

不要在没有确认载荷时向该目录写大量数据，否则可能写入本地根文件系统或触发错误位置。

④ [诊断点] 服务 `active`，但触发时报远端或权限错误

症状：`autofs.service` `active`，`map` 可展开，精确访问 `key` 报错。

判断：本地 `daemon` 已运行不代表 NFS location 可用。根据错误转入第 26 章的前置对象：名称解析、路由、远端导出、NFS 服务、来源限制和 UID/GID 权限。

下一条证据：核对 map 展开的实际 `host:/path`，再使用第 26 章的 NFS 证据。不要通过客户端挂载点 `chmod 777` 修复服务端拒绝；客户端目录权限不能改变服务器导出策略。

⑤ [诊断点] wildcard 中某个用户失败，其他用户成功

症状： `/rhome/alice` 可触发， `/rhome/bob` 失败。

当前假设：master、服务和 wildcard 总体机制基本成立，差异更可能位于替换后的远端目录或用户身份。

下一条证据：明确 & 对每个请求形成的远端路径，检查失败 key 的 journal 与 NFS 源。不要重写整个 master map；先修正单个远端对象或题目身份。

⑥ [诊断点] 修改配置后仍表现为旧关系

症状：文件内容已经更新，但触发仍指向旧源。

当前假设：修改的是错误文件、daemon 尚未 reload、direct/master 变更未刷新、旧载荷仍在使用，或失败 key 存在短暂负缓存。

下一条证据：

```
automount -m
systemctl reload autofs.service
findmnt -T /目标路径
journalctl -u autofs.service -b --no-pager
```

先退出使用旧载荷的进程，让其自然过期，再重新触发。不要在不知道引用者的情况下强制卸载。

⑦ [诊断点] 超过 timeout 仍未卸载

症状：显式设置 60 秒，等待更久后 NFS 载荷仍存在。

当前假设：目录并未空闲、Shell 当前目录在载荷中、文件仍打开、后台进程在扫描，或 timeout 未进入有效 master entry。

下一条证据：

```
automount -m
findmnt -T /目标路径
fuser -vm /目标路径
```

最小修复是释放活动引用或修正有效 timeout，然后重新计时；不是修改成更短时间并反复 restart。

⑧ [诊断点] SELinux 或其他安全证据出现时不关闭保护机制

若 journal 或审计日志出现访问控制证据，保留当前 map、源、身份和错误时间，转入第 28 章的 AVC 分析。关闭 SELinux 只能改变症状，不能证明策略层根因，也会破坏考试和真实工作的安全边界。

[Cheatsheet] 先问 entry 是否有效、key 是否匹配、是否真的触发、源是否正确、身份是否有权、载荷是否 busy；每次只修最小层，再从 `automount -m` 开始复验。

[知识专题] 从考试配置迁移到日常运维

考试环境通常只有少量主机和明确路径，真实系统则可能已有站点 master、LDAP/SSSD map、数百个用户目录和长时间运行的进程。工作迁移的关键不是引入更多命令，而是保持对象和证据可审计：变更前保存有效 map，使用独立 drop-in，控制 wildcard 范围，明确 timeout，并把配置正确与远端业务可用分开监控。

① [工作迁移] 变更前保存有效配置视图

在生产变更单中同时记录：

```
automount -m
systemctl status autofs.service --no-pager
findmnt -t autofs
findmnt -t nfs,nfs4
```

这样可区分“文件被改了”与“daemon 实际加载了什么”。对已有站点配置，不应直接清空 `/etc/auto.master`；使用独立 `.autofs` 片段，并检查同一 indirect mount point 是否已经定义。

② [工作迁移] wildcard 是扩展能力，也是故障范围

wildcard 可减少条目数量，但任何拼错 key 都会尝试形成一个远端路径。在用户规模较大时，应配合服务端目录生命周期、身份来源和监控设计，而不是把所有未知路径都当作正常。对少量异构共享，显式 key 通常更清楚。

③ [工作迁移] 监控应观察“能否触发”和“是否长期卡住”

只监控 `autofs.service active` 容易漏掉远端失败。更有价值的检查包括：选定代表 key 的只读访问、当前源是否正确、触发耗时、mount 失败日志，以及载荷是否长期保持 busy。监控脚本必须避免高频访问导致挂载永远不空闲。

④ [帮助入口] 从机制到参数逐层查找

```
man 5 auto.master
man 5 autofs
man 8 automount
man 8 autofs
systemctl help autofs.service
```

`auto.master(5)` 解释 master entry 和 direct/indirect 路径；`autofs(5)` 解释子 map 格式、wildcard 与 location；`automount(8)` 解释 daemon 和参数；`autofs(8)` 解释服务控制及 reload 行为。

[Cheatsheet] 生产变更保存有效视图；drop-in 优于清空主文件；wildcard 控制范围；监控真实触发而非只看 active；机制疑问回到四个手册入口。

[经典任务] 任务一：按用户 key 自动挂载远端家目录

环境与当前状态

你在示例客户端 `servera` 上工作。第 26 章已经确认远端 NFS 服务可用，并提供：

```
serverb.lab.example.com:/rhome/remotouser0
```

本机已经存在用户 `remotouser0`，其家目录应为：

```
/rhome/remotouser0
```

当前没有为 `/rhome` 配置本题 `autofs` map。目标路径不应写入 `/etc/fstab`，也不得通过普通手工 NFS 挂载冒充终态。

目标终态

1. 安装并启用需要的客户端组件；
2. 使用 `/etc/auto.master.d/rhome.autofs` 注册 indirect map；
3. 使用 `/etc/auto.rhome` 建立同名用户目录 wildcard 规则；
4. 挂载选项包含 `rw`，显式 timeout 为 60 秒；
5. 访问 `/rhome/remotouser0` 时挂载 `serverb.lab.example.com:/rhome/remotouser0`；
6. `autofs.service` 当前运行且开机启用；
7. `remotouser0` 能按题目要求读写；
8. 目录空闲后载荷可以卸载，再次访问能够重新触发。

限制条件

- 不修改远端 `/etc/exports`；
- 不使用 `chmod 777`；
- 不关闭 SELinux 或防火墙；
- 不删除现有站点 `autofs` 配置；
- 不把 `systemctl` 成功作为最终验收；
- 不编造命令输出，按当前系统实际证据判断。

验收证据

```
配置: automount -m
服务当前: systemctl is-active autofs
服务持久: systemctl is-enabled autofs
触发层: findmnt -t autofs
触发: stat /rhome/remotouser0
当前源: findmnt -T /rhome/remotouser0
功能: 以 remotouser0 验证读写
生命周期: 空闲后载荷消失, 再次访问恢复
```

请先独立完成。参考解答从下一页开始。

[参考解答] 任务一：调查、配置与分层验收

① [调查] 建立变更前基线

```
rpm -q autofs nfs-utils
automount -m
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
findmnt -t nfs,nfs4
getent hosts serverb.lab.example.com
ls -ld /rhome
```

若 `/rhome` 已被其他 master entry 管理, 不应再创建重复 indirect entry; 先调查站点配置并在现有 map 中做最小修改。NFS 导出与身份问题归第 26 章, 本题不修改服务器端。

② [操作] 安装组件并创建 master 片段

```
dnf install autofs nfs-utils

cat > /etc/auto.master.d/rhome.autofs <<'EOF'
/rhome /etc/auto.rhome --timeout=60
EOF
```

③ [操作] 创建 wildcard 子 map

```
cat > /etc/auto.rhome <<'EOF'
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
EOF
```

访问 key `remotouser0` 时, `&` 被替换为 `remotouser0`, 得到题目指定的远端源。

④ [静态验证] 先确认 map 关系

```
automount -m
```

确认 `/rhome`、`/etc/auto.rhome`、wildcard、location 和 timeout 均位于有效视图。若这一层错误，不进入服务 restart 猜测。

⑤ [加载] 建立服务当前与持久状态

```
systemctl enable --now autofs.service
systemctl reload autofs.service
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
```

首次 `enable --now` 后再 reload 并非绝对必要，但在参考解答中用于明确“配置已经按最终内容重新读取”。实际环境可在确认首次启动发生于配置写入之后时省略重复 reload。

⑥ [触发与当前源] 精确访问 key

```
stat /rhome/remoteuser0
findmnt -T /rhome/remoteuser0
findmnt -t nfs,nfs4 -o TARGET,SOURCE,FSTYPE,OPTIONS
```

核对远端源必须是 `serverb.lab.example.com:/rhome/remoteuser0`，而不是仅看到任意 NFS 条目。

⑦ [功能] 使用目标身份

```
sudo -u remoteuser0 test -r /rhome/remoteuser0
sudo -u remoteuser0 test -w /rhome/remoteuser0
sudo -u remoteuser0 touch /rhome/remoteuser0/.autofs-client-test
sudo -u remoteuser0 rm /rhome/remoteuser0/.autofs-client-test
```

若失败，保存错误和源关系，调查远端所有者、UID/GID 与导出策略；不在客户端挂载点使用 `chmod 777`。

⑧ [生命周期] 空闲、卸载与再触发

```
cd /
# 确认没有 Shell 或程序继续使用目标路径，并等待超过 60 秒
findmnt -t nfs,nfs4 -o TARGET,SOURCE

stat /rhome/remoteuser0
findmnt -T /rhome/remoteuser0
journalctl -u autofs.service -b --no-pager
```


静态核对环境无法承诺真实环境一定在精确第 60 秒完成卸载；若仍存在，先使用 `fuser -vm` 查活动引用。

典型错误

- master 写成 `/rhome/remotouser0 /etc/auto.rhome`，导致路径模型错误；
- indirect 子 map key 写成完整 `/rhome/remotouser0`；

典型错误（续）

- `.autofs` 文件扩展名或子 map 路径拼错；
- 只执行 `ls /rhome`，没有精确访问 key；
- `automount -m` 成功后直接宣布任务完成；
- root 能写就认为 `remotouser0` 一定能写；
- 为解决远端权限失败执行 `chmod 777` 或关闭 SELinux；
- Shell 停留在目标目录中却期待 timeout 卸载。

[经典任务] 任务二：map 可见但 `/projects/alpha` 不触发

已知证据

```
autofs.service 为 active 且 enabled
automount -m 能看到: /projects → /etc/auto.projects
findmnt -t autofs 能看到 /projects
访问 /projects/alpha 后没有对应 NFS 载荷
远端源 files.example.com:/exports/projects/alpha 已由第 26 章确认可用
```

当前 `/etc/auto.projects` 为：

```
/projects/alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
```

目标

在不修改远端服务器、不删除其他 autofs 配置、不使用普通手工挂载替代的前提下，定位并修复最小错误，使 `/projects/alpha` 能按需挂载，随后用配置、触发、当前源和超时再触发完成验收。

请先独立完成。参考解答从下一页开始。

[参考解答] 任务二：比较 master 类型与 key 形式

① [调查] 利用已有证据缩小范围

服务 active、master entry 可见且 `/projects` 触发层存在，说明软件包、daemon 与 master point 大体成立。远端源也已独立确认，下一条最有区分度的证据不是 restart，而是比较 indirect 路径模型：

```
master mount point: /projects
请求路径: /projects/alpha
请求 key: alpha
当前 map key: /projects/alpha
```

当前子 map 把完整路径写成了 indirect key，因此 key 不匹配。

② [最小修复] 只改第一字段

```
cat > /etc/auto.projects <<'EOF'
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
EOF
```

③ [加载与再验证]

```
automount -m
systemctl reload autofs.service
stat /projects/alpha
findmnt -T /projects/alpha
journalctl -u autofs.service -b --no-pager
```

确认当前源与目标正确，再按题目身份执行读写测试。若需要验证 timeout，先退出目标目录并清除活动引用，等待后观察载荷，再次 `stat` 确认能够恢复。

④ [为什么其他修复不合适]

- restart 不会自动把错误 key 改成 `alpha`；
- 修改远端 `/exports` 与已知证据矛盾；
- 在 `/etc/fstab` 中写永久挂载改变了题目生命周期；
- `mount -t nfs` 只能证明远端可挂载，不能修复 autofs key；
- 删除 `/projects` 目录不改变 map 匹配规则；
- 关闭安全机制或 `chmod 777` 与本层错误无关。

[本章收束] 自动挂载的终态是一条可重复触发的证据链

完成 autofs 任务时，不要把目标写成“目录存在”或“服务已经启动”。真正的终态是：路径模型正确，master 与子 map 能进入有效配置视图，daemon 当前运行且按题意持久启用，精确 key 能建立正确载荷，目标身份能够完成所需访问，载荷空闲后可以卸载，并能在下一次访问时重新出现。

工作方法

识别 direct / indirect

→ 建立 master 与子 map

→ automount -m 静态展开

→ enable 或 reload

→ stat 精确 key

→ findmnt 核对触发层与载荷源

→ 目标身份功能验证

→ 空闲卸载与再次触发

任何一步失败，都先给它定位到 entry、key、trigger、remote、permission 或 busy 层，再选择一条最有区分度的证据。反复 restart、永久手工挂载、全局放宽权限和关闭安全机制，都不能替代对象判断。

主要判断表

看到的证据	可以证明	仍然不能证明
配置文件存在	管理员写入了目标配置	文件已被 master 加载、字段正确
automount -m 能展开 entry	master 与子 map 当前可解析	daemon 已 reload、远端能挂载
autofs.service 为 active	automount daemon 当前运行	指定 key 存在、载荷源正确
findmnt -t autofs 有结果	触发层已经建立	对应 NFS 载荷当前存在
精确 stat /base/key 成功	路径解析与触发至少可继续	当前源一定符合题目、目标用户可写
findmnt -T PATH 源正确	当前路径落在目标载荷上	timeout 正常、重启后仍可用
空闲后 NFS 载荷消失	自动卸载可能正常发生	触发能力仍然有效
再次访问重新出现	按需生命周期可以恢复	所有远端权限与业务语义均正确

向下一章交接

本章只负责 autofs 的路径、map、触发和生命周期。如果当前源已经正确，但访问仍因权限或安全策略失败，应保留 findmnt、目标身份和错误日志证据：NFS 导出、UID/GID 与远端身份问题回到第 26 章；出现 SELinux AVC 时进入第 28 章，用上下文与 AVC 证据继续定位，而不是关闭 SELinux。

静态验证边界：本章依据 RHEL 9 课程、细分课件和手册页完成静态核对；当前会话没有可控制的 RHEL 9 VM，未执行真实 map reload、NFS 触发、busy 引用、timeout 与冷启动测试。